

Note

在 C++ 语言中，名字查找发生在类型检查之前。

第一个调用传入一个字符串面值，但是当前作用域内 `print` 函数唯一的声明要求参数是 `int` 类型。字符串面值无法转换成 `int` 类型，所以这个调用是错误的。在外层作用域中的 `print(const string&)` 函数虽然与本次调用匹配，但是它已经被隐藏掉了，根本不会被考虑。

当我们为 `print` 函数传入一个 `double` 类型的值时，重复上述过程。编译器在当前作用域内发现了 `print(int)` 函数，`double` 类型的实参转换成 `int` 类型，因此调用是合法的。

假设我们把 `print(int)` 和其他 `print` 函数声明放在同一个作用域中，则它将成为另一种重载形式。此时，因为编译器能看到所有三个函数，上述调用的处理结果将完全不同：

```
void print(const string &);  
void print(double);           // print 函数的重载形式  
void print(int);              // print 函数的另一种重载形式  
void fooBar2(int ival)  
{  
    print("Value: ");         // 调用 print(const string &)  
    print(ival);              // 调用 print(int)  
    print(3.14);              // 调用 print(double)  
}
```

6.5 特殊用途语言特性

236

本节我们介绍三种函数相关的语言特性，这些特性对大多数程序都有用，它们分别是：默认实参、内联函数和 `constexpr` 函数，以及在程序调试过程中常用的一些功能。

6.5.1 默认实参

某些函数有这样一种形参，在函数的很多次调用中它们都被赋予一个相同的值，此时，我们把这个反复出现的值称为函数的**默认实参**（`default argument`）。调用含有默认实参的函数时，可以包含该实参，也可以省略该实参。

例如，我们使用 `string` 对象表示窗口的内容。一般情况下，我们希望该窗口的高、宽和背景字符都使用默认值。但是同时我们也应该允许用户为这几个参数自由指定与默认值不同的数值。为了使得窗口函数既能接纳默认值，也能接受用户指定的值，我们把它定义成如下的形式：

```
typedef string::size_type sz; // 关于 typedef 参见 2.5.1 节（第 60 页）  
string screen(sz ht = 24, sz wid = 80, char backgrnd = ' ');
```

其中我们为每一个形参都提供了默认实参，默认实参作为形参的初始值出现在形参列表中。我们可以为一个或多个形参定义默认值，不过需要注意的是，一旦某个形参被赋予了默认值，它后面的所有形参都必须有默认值。

使用默认实参调用函数

如果我们想使用默认实参，只要在调用函数的时候省略该实参就可以了。例如，

screen 函数为它的所有形参都提供了默认实参，所以我们可以使用 0、1、2 或 3 个实参调用该函数：

```
string window;
window = screen(); // 等价于 screen(24, 80, ' ')
window = screen(66); // 等价于 screen(66, 80, ' ')
window = screen(66, 256); // screen(66, 256, ' ')
window = screen(66, 256, '#'); // screen(66, 256, '#')
```

函数调用时实参按其位置解析，默认实参负责填补函数调用缺少的尾部实参（靠右侧位置）。例如，要想覆盖 backgrnd 的默认值，必须为 ht 和 wid 提供实参：

```
window = screen(, , '?'); // 错误：只能省略尾部的实参
window = screen('?', 80, '');
```

需要注意，第二个调用传递一个字符值，是合法的调用。然而尽管如此，它的实际效果却与书写的意图不符。该调用之所以合法是因为 '?' 是个 char，而函数最左侧形参的类型 string::size_type 是一种无符号整数类型，所以 char 类型可以转换成（参见 4.11 节，第 141 页）函数最左侧形参的类型。当该调用发生时，char 类型的实参隐式地转换成 string::size_type，然后作为 height 的值传递给函数。在我们的机器上，'?' 对应的十六进制数是 0x3F，也就是十进制数的 63，所以该调用把值 63 传给了形参 height。

当设计含有默认实参的函数时，其中一项任务是合理设置形参的顺序，尽量让不怎么使用默认值的形参出现在前面，而让那些经常使用默认值的形参出现在后面。

默认实参声明

对于函数的声明来说，通常的习惯是将其放在头文件中，并且一个函数只声明一次，但是多次声明同一个函数也是合法的。不过有一点需要注意，在给定的作用域中一个形参只能被赋予一次默认实参。换句话说，函数的后续声明只能为之前那些没有默认值的形参添加默认实参，而且该形参右侧的所有形参必须都有默认值。假如给定

```
// 表示高度和宽度的形参没有默认值
string screen(sz, sz, char = '');
```

我们不能修改一个已经存在的默认值：

```
string screen(sz, sz, char = '*'); // 错误：重复声明
```

但是可以按照如下形式添加默认实参：

```
string screen(sz = 24, sz = 80, char); // 正确：添加默认实参
```



通常，应该在函数声明中指定默认实参，并将该声明放在合适的头文件中。

默认实参初始化

局部变量不能作为默认实参。除此之外，只要表达式的类型能转换成形参所需的类型，该表达式就能作为默认实参：

```
// wd、def 和 ht 的声明必须出现在函数之外
sz wd = 80;
char def = ' ';
sz ht();
string screen(sz = ht(), sz = wd, char = def);
```

```
string window = screen();    // 调用 screen(ht(), 80, '')
```

用作默认实参的名字在函数声明所在的作用域内解析，而这些名字的求值过程发生在函数调用时：

```
void f2()
{
    def = '**';                // 改变默认实参的值
    sz wd = 100;               // 隐藏了外层定义的 wd，但是没有改变默认值
    window = screen();         // 调用 screen(ht(), 80, '**')
}
```

我们在函数 f2 内部改变了 def 的值，所以对 screen 的调用将会传递这个更新过的值。另一方面，虽然我们的函数还声明了一个局部变量用于隐藏外层的 wd，但是该局部变量与传递给 screen 的默认实参没有任何关系。

6.5.1 节练习

238

练习 6.40: 下面的哪个声明是错误的？为什么？

```
(a) int ff(int a, int b = 0, int c = 0);
(b) char *init(int ht = 24, int wd, char bckgrnd);
```

练习 6.41: 下面的哪个调用是非法的？为什么？哪个调用虽然合法但显然与程序员的初衷不符？为什么？

```
char *init(int ht, int wd = 80, char bckgrnd = ' ');
(a) init();          (b) init(24,10);          (c) init(14, '**');
```

练习 6.42: 给 make_plural 函数（参见 6.3.2 节，第 201 页）的第二个形参赋予默认实参 's'，利用新版本的函数输出单词 success 和 failure 的单数和复数形式。

6.5.2 内联函数和 constexpr 函数

在 6.3.2 节（第 201 页）中我们编写了一个小函数，它的功能是比较两个 string 形参的长度并返回长度较小的 string 的引用。把这种规模较小的操作定义成函数有很多好处，主要包括：

- 阅读和理解 shorterString 函数的调用要比读懂等价的表达式容易得多。
- 使用函数可以确保行为的统一，每次相关操作都能保证按照同样的方式进行。
- 如果我们需要修改计算过程，显然修改函数要比先找到等价表达式所有出现的地方再逐一修改更容易。
- 函数可以被其他应用重复利用，省去了程序员重新编写的代价。

然而，使用 shorterString 函数也存在一个潜在的缺点：调用函数一般比求等价的表达式的值要慢一些。在大多数机器上，一次函数调用其实包含着一系列工作：调用前要先保存寄存器，并在返回时恢复；可能需要拷贝实参；程序转向一个新的位置继续执行。

内联函数可避免函数调用的开销

将函数指定为内联函数（inline），通常就是将它在每个调用点上“内联地”展开。假设我们把 shorterString 函数定义成内联函数，则如下调用

239 `cout << shorterString(s1, s2) << endl;`

将在编译过程中展开成类似于下面的形式

```
cout << (s1.size() < s2.size() ? s1 : s2) << endl;
```

从而消除了 `shorterString` 函数的运行时开销。

在 `shorterString` 函数的返回类型前面加上关键字 `inline`, 这样就可以将它声明成内联函数了:

```
// 内联版本: 寻找两个 string 对象中较短的那个
inline const string &
shorterString(const string &s1, const string &s2)
{
    return s1.size() <= s2.size() ? s1 : s2;
}
```

Note

内联说明只是向编译器发出的一个请求, 编译器可以选择忽略这个请求。

一般来说, 内联机制用于优化规模较小、流程直接、频繁调用的函数。很多编译器都不支持内联递归函数, 而且一个 75 行的函数也不大可能在调用点内联地展开。

constexpr 函数

constexpr 函数 (constexpr function) 是指能用于常量表达式 (参见 2.4.4 节, 第 58 页) 的函数。定义 `constexpr` 函数的方法与其他函数类似, 不过要遵循几项约定: 函数的返回类型及所有形参的类型都得是字面值类型 (参见 2.4.4 节, 第 59 页), 而且函数体中必须有且只有一条 `return` 语句:

C++
11

```
constexpr int new_sz() { return 42; }
constexpr int foo = new_sz(); // 正确: foo 是一个常量表达式
```

我们把 `new_sz` 定义成无参数的 `constexpr` 函数。因为编译器能在程序编译时验证 `new_sz` 函数返回的是常量表达式, 所以可以用 `new_sz` 函数初始化 `constexpr` 类型的变量 `foo`。

执行该初始化任务时, 编译器把对 `constexpr` 函数的调用替换成其结果值。为了能在编译过程中随时展开, `constexpr` 函数被隐式地指定为内联函数。

`constexpr` 函数体内也可以包含其他语句, 只要这些语句在运行时不执行任何操作就行。例如, `constexpr` 函数中可以有空语句、类型别名 (参见 2.5.1 节, 第 60 页) 以及 `using` 声明。

我们允许 `constexpr` 函数的返回值并非一个常量:

```
// 如果 arg 是常量表达式, 则 scale(arg) 也是常量表达式
constexpr size_t scale(size_t cnt) { return new_sz() * cnt; }
```

当 `scale` 的实参是常量表达式时, 它的返回值也是常量表达式; 反之则不然:

240

```
int arr[scale(2)]; // 正确: scale(2) 是常量表达式
int i = 2;         // i 不是常量表达式
int a2[scale(i)];  // 错误: scale(i) 不是常量表达式
```

如上例所示, 当我们给 `scale` 函数传入一个形如字面值 2 的常量表达式时, 它的返回类型也是常量表达式。此时, 编译器用相应的结果值替换对 `scale` 函数的调用。

如果我们用一个非常量表达式调用 `scale` 函数，比如 `int` 类型的对象 `i`，则返回值是一个非常量表达式。当把 `scale` 函数用在需要常量表达式的上下文中时，由编译器负责检查函数的结果是否符合要求。如果结果恰好不是常量表达式，编译器将发出错误信息。



constexpr 函数不一定返回常量表达式。

把内联函数和 constexpr 函数放在头文件内

和其他函数不一样，内联函数和 `constexpr` 函数可以在程序中多次定义。毕竟，编译器要想展开函数仅有函数声明是不够的，还需要函数的定义。不过，对于某个给定的内联函数或者 `constexpr` 函数来说，它的多个定义必须完全一致。基于这个原因，内联函数和 `constexpr` 函数通常定义在头文件中。

6.5.2 节练习

练习 6.43: 你会把下面的哪个声明和定义放在头文件中？哪个放在源文件中？为什么？

- (a) `inline bool eq(const BigInt&, const BigInt&) {...}`
- (b) `void putValues(int *arr, int size);`

练习 6.44: 将 6.2.2 节（第 189 页）的 `isShorter` 函数改写成内联函数。

练习 6.45: 回顾在前面的练习中你编写的那些函数，它们应该是内联函数吗？如果是，将它们改写成内联函数；如果不是，说明原因。

练习 6.46: 能把 `isShorter` 函数定义成 `constexpr` 函数吗？如果能，将它改写成 `constexpr` 函数；如果不能，说明原因。

6.5.3 调试帮助

C++ 程序员有时会用到一种类似于头文件保护（参见 2.6.3 节，第 67 页）的技术，以便有选择地执行调试代码。基本思想是，程序可以包含一些用于调试的代码，但是这些代码只在开发程序时使用。当应用程序编写完成准备发布时，要先屏蔽掉调试代码。这种方法用到两项预处理功能：`assert` 和 `NDEBUG`。

assert 预处理宏

assert 是一种预处理宏（preprocessor marco）。所谓预处理宏其实是一个预处理变量，它的行为有点类似于内联函数。`assert` 宏使用一个表达式作为它的条件：

```
assert (expr);
```

首先对 `expr` 求值，如果表达式为假（即 0），`assert` 输出信息并终止程序的执行。如果表达式为真（即非 0），`assert` 什么也不做。

`assert` 宏定义在 `cassert` 头文件中。如我们所知，预处理名字由预处理器而非编译器管理（参见 2.3.2 节，第 49 页），因此我们可以直接使用预处理名字而无须提供 `using` 声明。也就是说，我们应该使用 `assert` 而不是 `std::assert`，也不需要为 `assert` 提供 `using` 声明。

和预处理变量一样，宏名字在程序内必须唯一。含有 `cassert` 头文件的程序不能再定义名为 `assert` 的变量、函数或者其他实体。在实际编程过程中，即使我们没有包含

cassert 头文件，也最好不要为了其他目的使用 assert。很多头文件都包含了 cassert，这就意味着即使你没有直接包含 cassert，它也很可能通过其他途径包含在你的程序中。

assert 宏常用于检查“不能发生”的条件。例如，一个对输入文本进行操作的程序可能要求所有给定单词的长度都大于某个阈值。此时，程序可以包含一条如下所示的语句：

```
assert(word.size() > threshold);
```

NDEBUG 预处理变量

assert 的行为依赖于一个名为 NDEBUG 的预处理变量的状态。如果定义了 NDEBUG，则 assert 什么也不做。默认状态下没有定义 NDEBUG，此时 assert 将执行运行时检查。

我们可以使用一个#define 语句定义 NDEBUG，从而关闭调试状态。同时，很多编译器都提供了一个命令行选项使我们可以定义预处理变量：

```
$ CC -D NDEBUG main.C # use /D with the Microsoft compiler
```

这条命令的作用等价于在 main.c 文件的一开始写#define NDEBUG。

定义 NDEBUG 能避免检查各种条件所需的运行时开销，当然此时根本就不会执行运行时检查。因此，assert 应该仅用于验证那些确实不可能发生的事情。我们可以把 assert 当成调试程序的一种辅助手段，但是不能用它替代真正的运行时逻辑检查，也不能替代程序本身应该包含的错误检查。

除了用于 assert 外，也可以使用 NDEBUG 编写自己的条件调试代码。如果 NDEBUG 未定义，将执行#ifdef 和#endif 之间的代码；如果定义了 NDEBUG，这些代码将被忽略掉：

242

```
void print(const int ia[], size_t size)
{
#ifdef NDEBUG
    // __func__ 是编译器定义的一个局部静态变量，用于存放函数的名字
    cerr << __func__ << ": array size is " << size << endl;
#endif
    // ...
}
```

在这段代码中，我们使用变量__func__输出当前调试的函数的名字。编译器为每个函数都定义了__func__，它是 const char 的一个静态数组，用于存放函数的名字。

除了 C++ 编译器定义的__func__之外，预处理器还定义了另外 4 个对于程序调试很有用的名字：

- __FILE__ 存放文件名的字符串面值。
- __LINE__ 存放当前行号的整型面值。
- __TIME__ 存放文件编译时间的字符串面值。
- __DATE__ 存放文件编译日期的字符串面值。

可以使用这些常量在错误消息中提供更多信息：

```
if (word.size() < threshold)
    cerr << "Error: " << __FILE__
        << " : in function " << __func__
```

```
<< " at line " << __LINE__ << endl
<< "          Compiled on " << __DATE__
<< " at " << __TIME__ << endl
<< "          Word read was \"" << word
<< "\"": Length too short" << endl;
```

如果我们给程序提供了一个长度小于 threshold 的 string 对象, 将得到下面的错误消息:

```
Error:wdebug.cc : in function main at line 27
      Compiled on Jul 11 2012 at 20:50:03
      Word read was "foo": Length too short
```

6.5.3 节练习

练习 6.47: 改写 6.3.2 节 (第 205 页) 练习中使用递归输出 vector 内容的程序, 使其有条件地输出与执行过程有关的信息。例如, 每次调用时输出 vector 对象的大小。分别在打开和关闭调试器的情况下编译并执行这个程序。

练习 6.48: 说明下面这个循环的含义, 它对 assert 的使用合理吗?

```
string s;
while (cin >> s && s != sought) { } // 空函数体
assert(cin);
```

6.6 函数匹配



在大多数情况下, 我们容易确定某次调用应该选用哪个重载函数。然而, 当几个重载函数的形参数量相等以及某些形参的类型可以由其他类型转换得来时, 这项工作就不那么容易了。以下面这组函数及其调用为例:

```
void f();
void f(int);
void f(int, int);
void f(double, double = 3.14);
f(5.6);          // 调用 void f(double, double)
```

确定候选函数和可行函数

◀ 243

函数匹配的第一步是选定本次调用对应的重载函数集, 集合中的函数称为**候选函数** (candidate function)。候选函数具备两个特征: 一是与被调用的函数同名, 二是其声明在调用点可见。在这个例子中, 有 4 个名为 f 的候选函数。

第二步考察本次调用提供的实参, 然后从候选函数中选出能被这组实参调用的函数, 这些新选出的函数称为**可行函数** (viable function)。可行函数也有两个特征: 一是其形参数量与本次调用提供的实参数量相等, 二是每个实参的类型与对应的形参类型相同, 或者能转换成形参的类型。

我们能根据实参的数量从候选函数中排除掉两个。不使用形参的函数和使用两个 int 形参的函数显然都不适合本次调用, 这是因为我们的调用只提供了一个实参, 而它们分别有 0 个和两个形参。

使用一个 int 形参的函数和使用两个 double 形参的函数是可行的, 它们都能用一